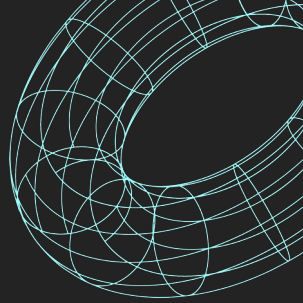




What-If Database

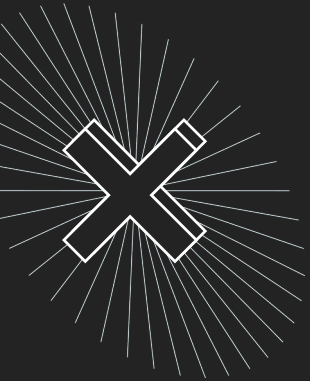
BACSE202 – Database Management Systems

Review 1

Prakriti Prakash Jha (25BDS0066)

Kahill Manuel Alvares (25BCE2873)

Himali Agarwal (25BAI0012)



Brief description of the project



Problem statement

Testing a database change means cloning it or hand-writing before/after queries, which is slow and risky on live data. Existing tools are analytical, look at the past, or need a branch first.



Proposed solution

A WHAT_IF layer on PostgreSQL runs the change inside a transaction, recomputes the metrics, then rolls back. Cascades and schema changes are reported exactly, not estimated.



Expected outcome

An impact report with affected tables, CGPA change and eligibility change, plus broken objects for schema changes. Success: correct reports on 6-7 demo scenarios with real data unchanged.



Key techniques in the domain

Hypothetical queries

Evaluate a query as if an update had been applied (Q when U). The idea behind every `WHAT_IF` statement.

Transactions & savepoints

Group statements atomically and undo them with `ROLLBACK`. Our non-destructive sandbox.

Snapshot isolation

Each transaction reads a consistent snapshot, keeping real data pristine during a simulation.



Query parsing

Turn custom syntax into standard SQL the engine can run. We parse `WHAT_IF` and validate it with `sqlglot`.

Dependency tracing

Find foreign keys, views and triggers that depend on an object. Used to report what schema changes break.

Temporal data

Keeps past states of data. Used for comparison: they look backward, we simulate forward.



Literature review

Ref.	Author(s), Year	Theme	Technique	Dataset / tools	Key findings	Limitations
[1]	Balmin et al., 2000	G1: Hypothetical queries	Hypothetical queries "Q when U"	OLAP / data warehouse	Formal semantics and efficient evaluation of what-if queries	Analytical warehouses only; not live transactional databases
[2]	Meliou & Suciu, 2012	G1: Hypothetical queries	Tiresias: how-to queries as an optimisation problem	Relational data with a solver	Finds the data changes needed to reach a goal (reverse what-if)	Answers the reverse question; solver-based and costly
[3]	Galhotra et al., 2022	G1: Hypothetical queries	HyperR: what-if and how-to with a probabilistic causal model	Relational data plus causal graph	Estimates the effect of hypothetical updates	Needs a causal model; estimates, not exact
[4]	Campbell et al., 2022	G1: Hypothetical queries	Mahif: historical what-if answered by slicing	Transaction histories	Answers "what if a past update had differed?" efficiently	Concerns past histories, not future changes



Literature review (continued)



Ref.	Author(s), Year	Theme	Technique	Dataset / tools	Key findings	Limitations
[5]	DoltHub, n.d.	G2: Versioned data	Dolt: SQL database with Git-style branch, diff, merge	Version-controlled SQL database	Copy-on-write branches let you try changes safely	Branches managed by hand; no automatic impact report
[6]	Berenson et al., 1995	G3: Isolation	Isolation levels, including snapshot isolation	ANSI SQL isolation levels	Each transaction gets a consistent view	A concurrency mechanism, not a what-if tool
[7]	Kulkarni & Michels, 2012	G2: Versioned data	SQL:2011 temporal tables	System-versioned / application-time tables	Standard way to query data at an earlier time	Looks backward only; cannot apply a hypothetical change

Research gaps

G1: Live databases

No one-step what-if on a live operational database. Work targets warehouses [1] or needs a branch first [5].

G3: Looks backward

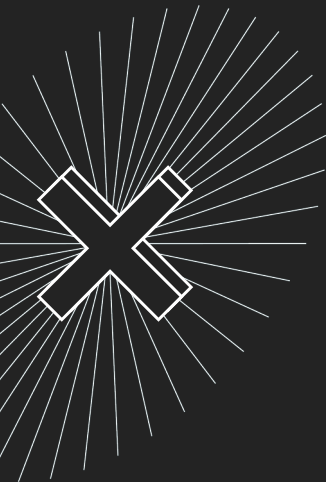
Historical what-if [4] and temporal tables [7] look at the past; snapshot isolation [6] is a mechanism, not a user feature.

G2: Exact vs estimates

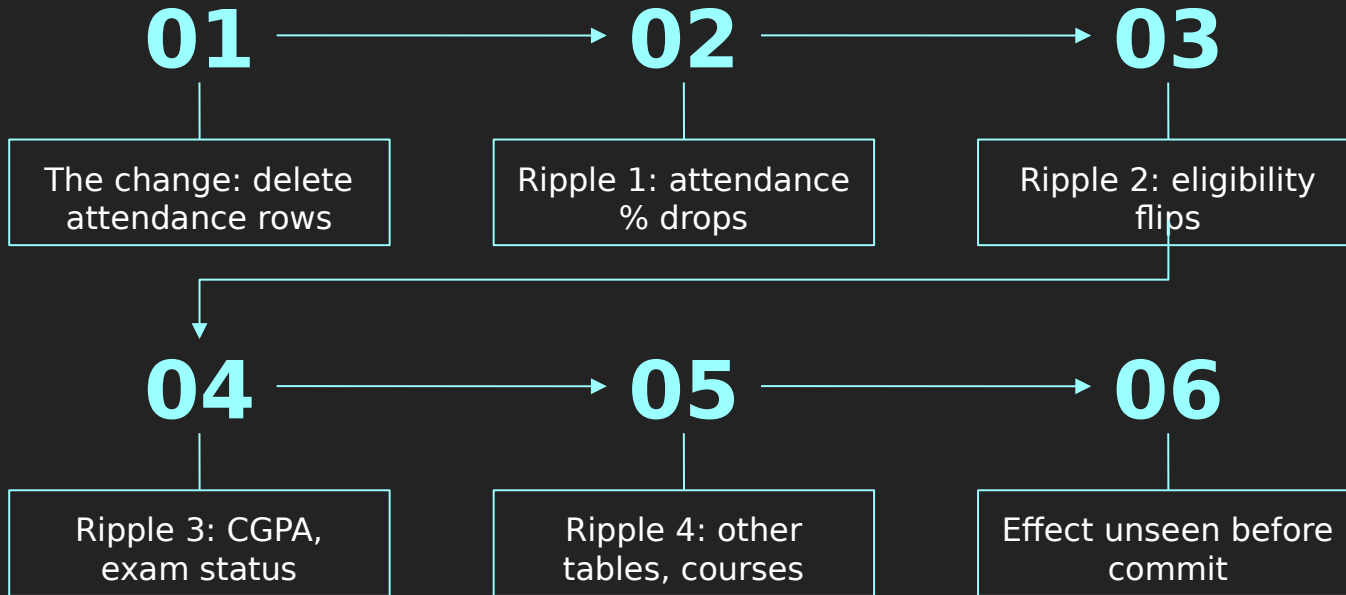
Causal methods give estimates [3] and how-to methods solve the reverse problem [2]. Exact simulation of cascades is missing.

G4: Schema changes

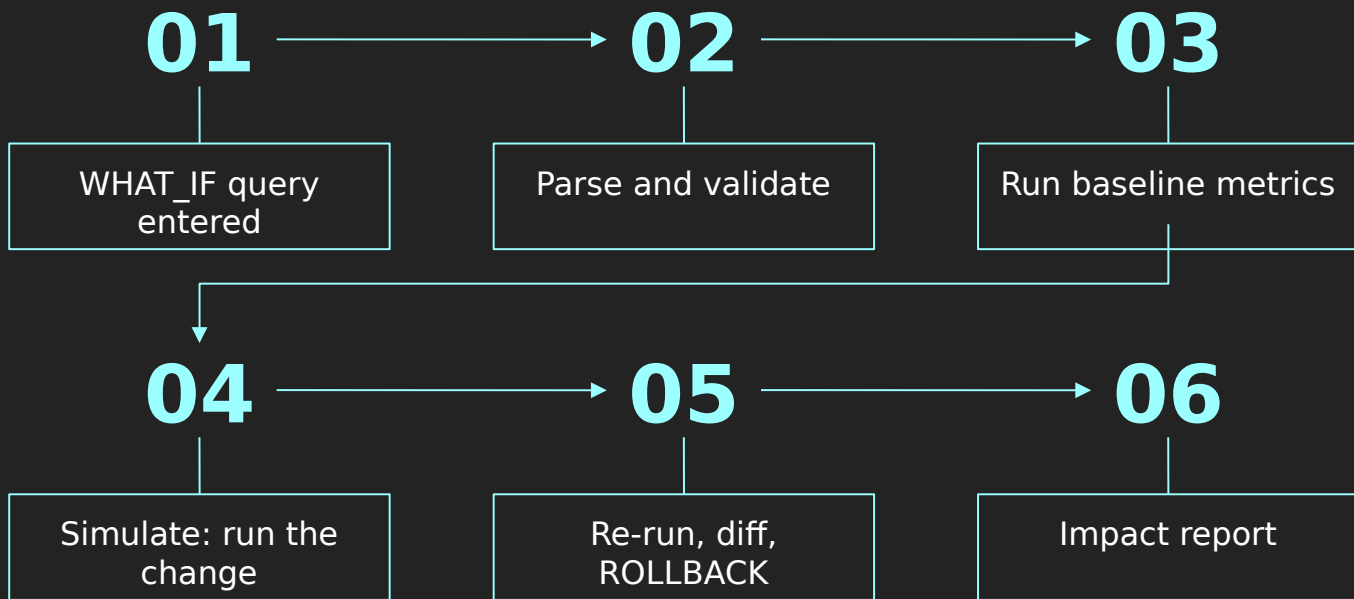
None of [1]-[7] reports which views, constraints and metrics break if a column or table is dropped.



The ripple effect

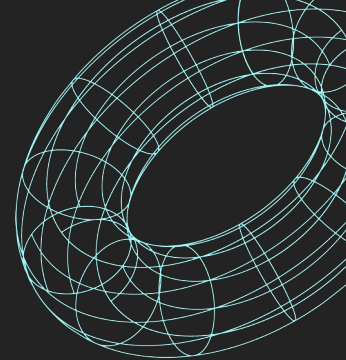


How What-If DB handles it



Steps 3 to 5 run inside one transaction: BEGIN ... ROLLBACK





Objectives

Main objective: build a lightweight layer that runs hypothetical changes in a rolled-back transaction and reports their exact impact on data and schema.

O1: Parse WHAT_IF statements and run them in an always-rolled-back sandbox.

O2: Report before/after metrics (CGPA, attendance, eligibility), including cascades.

O3: List the views, constraints and metrics that break under a schema change.



References (APA)

- [1] Balmin, A., Papadimitriou, T., & Papakonstantinou, Y. (2000). Hypothetical queries in an OLAP environment. VLDB.
- [2] Meliou, A., & Suciu, D. (2012). Tiresias: The database oracle for how-to queries. ACM SIGMOD.
- [3] Galhotra, S., Gilad, A., Roy, S., & Salimi, B. (2022). HypeR: Hypothetical reasoning with what-if and how-to queries using a probabilistic causal approach. ACM SIGMOD.
- [4] Campbell, F. S., Arab, B. S., & Glavic, B. (2022). Efficient answering of historical what-if queries. ACM SIGMOD.
- [5] DoltHub. (n.d.). Dolt: Git for data. <https://github.com/dolthub/dolt>
- [6] Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., & O'Neil, P. (1995). A critique of ANSI SQL isolation levels. ACM SIGMOD.
- [7] Kulkarni, K., & Michels, J.-E. (2012). Temporal features in SQL:2011. ACM SIGMOD Record, 41(3).